International Conference on Intelligent Systems and Machine Learning (ICISML-2026)

Context-Aware Vehicle Insurance Assessor Using a Multi-Modal ML Pipeline with Visual Damage Classification and Ensemble Claim Regression

Dr. Sugumar R[1], G K Mahanidhi[2], Mohammed Sohail[3], Mohammed Ismail[4], Mohamed Ajmal H[5]

Department of Computer Science and Engineering
Jain University, Bengaluru, Karnataka, India

GitHub Repository: https://github.com/Mahanidhii/Project-Centric-Learning

Abstract—The V.I.E.W. System is a machine learning pipeline that helps with vehicle insurance claims. It looks at the damage to a vehicle. Figure out how much money the insurance company should pay. The system uses a kind of computer program to look at pictures of the damage and another program to look at information about the claim. These two programs work together to come up with an estimate of the damage. The picture program uses something called EfficientNet B0 to look at the damage. It is very good at it. It can correctly identify what is wrong with the vehicle 84 percent of the time. The other program uses something called CatBoost Regressor to look at information about the claim. It is also very good. It can figure out how money the insurance company should pay with a pretty small amount of error. The system also has a set of rules that it uses to make sure the estimates are good. These rules take into account things like the kind of vehicle and how old it is. The system is easy to use. It can show the user all the information they need to see. It can even show them a chart that breaks down the estimate into smaller parts. The V.I.E.W. System is better than systems that do the same thing. It is more accurate. It can give answers more quickly. The system is a tool for insurance companies to use when they are figuring out how much to pay for vehicle damage. It uses machine learning. It is very helpful. The V.I.E.W. System is a way to make sure that insurance claims are handled correctly and that everyone is happy with the result. It is an example of how machine learning can be used to make things better. The V.I.E.W. System is a machine learning pipeline that helps with vehicle insurance claims. It is very good, at its job.

I. Introduction

The vehicle insurance industry deals with hundreds of millions of damage claims every year. With all the advances in technology the first step in this process, which is checking the damage to a vehicle is still done by a person. A person has to look at the damaged vehicle figure out what kind of damage it is and how bad it is check the insurance policy and then make an estimate of how much it will cost to fix it. This process takes a time usually around 7 to 15 working days and it can be different depending on who is doing the checking. It also does not work well when there are a lot of claims at the time like after a big disaster. Nowadays people have good cameras on their phones so we have a lot of pictures of damaged vehicles. At the time computers are getting better at looking at pictures and figuring out what is in them. This means we can use computers to look at pictures of damaged vehicles and figure out how bad the damage is. We can also use computers to look at insurance claims and make predictions about how much new claims will cost. The problem is that we do not have a system that can do both of these things together. We have systems that can look at pictures and figure out how bad the damage is and we have systems that can make predictions about how much claims will cost.. We do not have a system that can do both of these things and then use the information from the pictures to make the predictions. This means that insurance people have to enter the information into different systems, which can lead to mistakes.

This paper is about a system called V.I.E.W. It is a system that can look at pictures of damaged vehicles figure out how bad the damage is and then use that information to make a prediction about how much the claim will cost. The system has a few parts, including:

- A computer program that can look at pictures of damaged vehicles and figure out how bad the damage is. This program is really good at its job. Can even beat other programs that are commonly used.
- A computer program that can look at insurance claims and make predictions about how much new claims will cost. This program is also really good at its job. Can even beat other programs that are commonly used.
- A system that combines the predictions from the computer program with some rules that insurance people use to make estimates. This system is really good at making sure the estimates are accurate and reliable.
- A way for the system to remember what it has already done so insurance people do not have to enter the information into different parts of the system.
- A way for the system to show insurance people how it made its estimates, which's important for making sure everything is fair and transparent.

The rest of this paper is organized into sections. Section II looks at what other people have done in this area. Section III talks about what the system needs to be able to do. Section IV looks at the problems, with existing systems. Section V shows how the new system works. Section VI talks about the data that was used to train the system. Section VII explains how the computer program that looks at pictures was trained. Section VIII explains how the computer program that makes predictions was trained. Section IX shows what the system looks like to the people who use it. Section X talks about the results of some tests that were done on the system. Section XI explains why the system was designed in a way. Section XII sums up what was learned and what needs to be done.

II. Related Work and Literature Survey

A. Object Detection and Traditional Damage Assessment

Early automated damage detection systems used things like Scale-Invariant Feature Transform and Histogram of Oriented Gradients. These methods worked okay when things were controlled. They did not work well when the lighting was bad or when things were in the way. They also had a time with all the different types of damage that can happen to vehicles. Then deep learning came along. Changed things. Region-based CNN architectures like R-CNN and Faster R-CNN were able to find damage in an area. They could do this by predicting a box around the damage.. To train these systems you had to label every single pixel, which took a lot of time and work. This was a problem because it was hard to get the data we needed. Also it took two steps to get an answer, which was too slow for the web.

The automated damage detection systems relied on Scale-Invariant Feature Transform and Histogram of Oriented Gradients.. The automated damage detection systems did not work well with real world problems. The transition to learning helped the automated damage detection systems. The Region-based CNN architectures, like R-CNN and Faster R-CNN helped the automated damage detection systems find damage. The automated damage detection systems used Region-based CNN architectures to find the damage.

B. Single-Stage Detection Architectures

You Only Look Once architectures are really good at finding things in one pass, which makes them a lot faster than methods that need multiple steps. When people used You Only Look Once for checking vehicle damage it worked well and found most of the damage. One study used ten You Only Look models together and they were trained on a special dataset that was labelled by people who work for insurance companies. This showed that using models together can be a good way to get accurate results that are good enough for insurance companies. Some people have compared You Only Look Once version 5. You Only Look Once version 8. Found that they have some trade-offs like finding more things but also finding some things that are not really there which can affect whether or not an insurance claim is accepted. The problem with these methods is that they need a lot of information about each area of the picture and they need powerful computers to work. You Only Look Once methods are still really useful for things, like checking vehicle damage but they have some limitations.

C. Transfer Learning for Classification

Using a model that was first trained on ImageNet makes a difference. It means we need a lot data that is specific to what we are trying to do. This makes it possible to use the model with sets of data that have been labelled. The MobileNetV2 and EfficientNet and ResNet models have been used to classify whether or not a vehicle is damaged. The EfficientNet model is very good because it can be made bigger in a few ways. This means it can be as accurate as models but it has a lot fewer parts. This makes it very good for things that need to happen on the web. There is a way to make these models work better with data. First we do not change the part of the model. Then we make some changes to it. This helps the model remember what it already knew. It also helps it learn the things we want it to know. The EfficientNet model is very good, at this.

D. Insurance Claim Prediction Models

The old way of guessing how much a claim will cost was to use math models that looked at a few important things and gave them more weight. These models thought that the things that affect the cost of a claim are related in a way but that is not true. In life things like what kind of car someone has what kind of fuel it uses and how it crashed are all connected in complicated ways. Newer methods like Gradient-boosted ensembles, which include Random Forest, XGBoost, LightGBM and CatBoost are better at figuring out how much a claim will cost when we have a lot of information about it. CatBoost is especially good when we have a lot of information that can be divided into categories like the type of car or the time of day. It helps to make sure that we are not just guessing randomly by using tricks like ordered target statistics for categorical features and ordered boosting to reduce variance. This is really helpful when we have an amount of information and a lot of it is, about categories.

E. Research Gap

People have noticed that when they look at studies they always separate checking for damage from figuring out how much money is involved. These are two parts of the process that happens in real life. There is no system there that combines these two parts in a simple way, where a computer can look at pictures and then use that information to estimate costs and also check that it is correct all, within one website that people can use. V.I.E.W. Is trying to fix this problem.

III. System Requirements

A. Software Configuration

TABLE I: Software Configuration

Component	Minimum	Recommended
Operating System	Windows 10 / Ubuntu 18.04+	Windows 11 / Ubuntu 22.04 LTS
Python Version	Python 3.8+	Python 3.10 (tested)
Deep Learning	TensorFlow 2.10+	TensorFlow 2.21.0 + Keras
Gradient Boosting	CatBoost 1.0+	CatBoost 1.2.10
Web Framework	Streamlit 1.40+	Streamlit 1.56.0
Data Processing	Pandas 1.5+, NumPy 1.24+	Pandas 3.0.2, NumPy 2.4.4
Visualization	Plotly 5.x+	Plotly 6.7.0

B. Hardware Configuration

TABLE II: Hardware Configuration

Component	Minimum	Recommended
CPU	Intel Core i5 10th Gen	Intel Core i7/i9 / AMD Ryzen 9
GPU	NVIDIA GTX 1650 (4 GB VRAM)	NVIDIA RTX 3060 / Tesla A100
RAM	8 GB DDR4	16 GB – 32 GB DDR4/DDR5
Storage	256 GB SSD + 1 TB HDD	512 GB – 1 TB NVMe SSD

C. Python Dependency Stack

We use a Python environment to manage all the things our project needs. This helps keep everything separate and makes sure we get the results every time. We picked versions of some important tools that work well together and we made sure they do by testing them all together: TensorFlow 2.21.0 and Keras, CatBoost 1.2.10 and Pandas 3.0.2, NumPy 2.4.4, Scikit-learn 1.8.0, Pillow 12.2.0, Plotly 6.7.0, Matplotlib 3.10.9, Seaborn 0.13.2, and Streamlit 1.56.0.

IV. Existing Systems and Their Limitations

A. Heavyweight CNN Architectures (VGG16 / ResNet-50)

Early deep learning methods used models like VGG16, which has 138 million parameters, and ResNet-50, which has 25 million parameters. These models are good at getting accuracy on large datasets. However, they have some problems that make them hard to use for insurance. They take long to make predictions, which is a problem for websites that need to respond quickly. The model files are big. VGG16 is about 528 megabytes. Which takes up a lot of storage space. These models do not work well when the data is different from what they were trained on. This is especially true for

insurance photos that are taken by users, which often have problems like lighting or things blocking the view. VGG16 and ResNet-50 models struggle with these issues. They are not ideal for insurance deployment due, to these limitations.

B. Region-Based Detection Systems (Faster R-CNN)

Region-based architectures are good at finding damage in an area by predicting where the damage is located in a box. This gives us information about what is wrong than just saying what type of damage it is.. To get this to work we need people to draw boxes around the damage on each picture, which takes a lot of time and effort. We need to do this for every area that is damaged. This is much harder than labeling the whole picture. Also the way we use these architectures to make predictions is slow. That means we cannot use them in real time for things that need to happen quickly. Region-based architectures are used for this. They have some problems. Region-based architectures need these boxes to be drawn by people and that is why they are not very good, for things that need to be fast.

C. Commercial Proprietary Platforms

Major insurance companies are using Enterprise AI appraisal tools to show that automated damage assessment can be commercially viable. However these Enterprise AI appraisal tools are not open, to everyone. They are closed systems that only big insurance companies can use because they have to buy a license. This means that smaller insurance companies and people who work on their own cannot use these Enterprise AI appraisal tools. The Enterprise AI appraisal tools are also connected to databases that have the costs of repairs and these are owned by the companies that made the Enterprise AI appraisal tools. The way that the Enterprise AI appraisal tools figure out how much damage is done is not clear so it is hard for researchers to study them or for governments to check if they are working correctly. This also means that it is not possible for other people to use the Enterprise AI appraisal tools on their own.

D. Statistical Regression for Claim Prediction

The old way of using linear regression models to figure out insurance claims is not very good. These models think that the relationship between what happens and the cost is always the same.. That is not true. Sometimes things do not work out that way. For example if you have a nice car and it gets a small dent it can cost a lot to fix.. If you have a cheaper car with the same kind of damage it does not cost as much to fix. The old models do not understand this unless you tell them to look for these kinds of things. Linear regression has trouble with things like this. The traditional actuarial linear regression models do not work well for insurance claims, like the premium vehicle and the budget vehicle.

V. System Architecture

V.I.E.W. Works in two phases. It uses two machine learning models and combines their results. Phase 1 looks at a photo of the vehicle damage. It uses a model called EfficientNet-B0 to classify the damage and estimate how severe it is. Phase 2 asks for information about the vehicle and what happened. It uses another model, CatBoost to make a prediction. This prediction is converted to Rupees. The prediction is then combined with an actuarial estimate. This combination is done through something called the Hybrid Estimation Engine. The final result is an estimate of the claim amount, in Rupees. There is also a range of values, which is plus or minus 15 percent. The breakdown of the estimate is shown in a itemised format.

TABLE III: System Component Overview

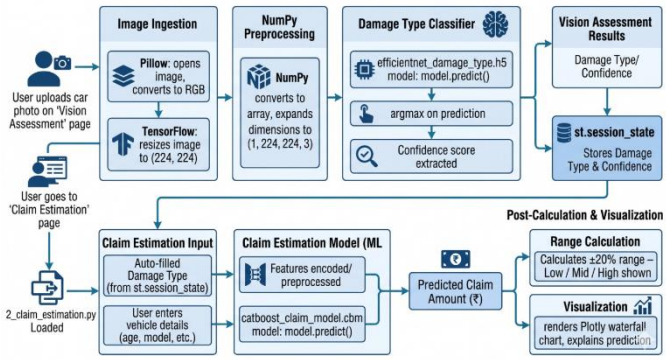


Figure 1:System Architecture

Component	Technology	Purpose
Vision Classifier	EfficientNet-B0 (TensorFlow/Keras)	Classifies damage type from uploaded images
Regression Predictor	CatBoost Regressor	Predicts vehicle claim from tabular features
Tier Classifier	CatBoost Classifier	Validates tier (Low/Medium/High) via macro-F1
Hybrid Engine	Custom Python (Rule-Based + ML Blend)	Produces context-aware financial estimate
Web Interface	Streamlit (Multi-page)	Interactive dashboard for end-users
Visualization	Plotly (Waterfall Charts)	Transparent cost breakdown display

A. Phase Interconnection

The damage class we found in Phase 1 is automatically carried over to Phase 2 through Streamlits session state so users don't have to enter it This automatic carry-over is important because it helps keep things consistent. The damage class helps us pick the Base Cost in the rule-based estimate. Is also used as a category in the CatBoost regressor. This close connection makes sure that what we see directly affects both the machine learning and rule-based results, which helps keep the classification and estimation parts consistent. The damage class is really important here. We use it to make sure both parts of our system work well together. It is used in both the Base Cost selection and as an input, to the CatBoost regressor.

VI. Dataset Description

A. Visual Dataset — VehiDE

The vision module is trained on VehiDE (Vehicle Damage Estimation), a benchmark dataset comprising approximately 2 GB of annotated vehicle damage images captured under diverse real-world conditions including varied lighting, occlusion, viewing angles, and damage morphologies. Damage regions are annotated at instance level using polygon masks stored in VIA-format JSON: the training annotation file (~35 MB) and validation annotation file (~7 MB).

A custom preprocessing pipeline (dataset_parser.py) converts instance-level segmentation annotations into a standard image classification dataset. The pipeline: (1) parses VIA-format JSON to extract polygon coordinates per damage region; (2) computes axis-aligned bounding boxes from polygon vertices; (3) applies 10% spatial padding around each bounding box to preserve contextual information at region boundaries; and (4) saves cropped damage patches into class-specific subdirectories compatible with TensorFlow's image_dataset_from_directory utility.

TABLE IV: VehiDE Damage Class Taxonomy

Raw Class	English Translation	Severity Tier
tray_son	Paint Scratch / Surface Scuff	Minor
rach	Deep Scratch / Laceration	Minor
mop_lom	Dented Body Panel	Moderate
be_den	Cracked / Broken Headlight	Moderate
vo_kinh	Shattered Glass	Major
thung	Punctured / Holed Panel	Major
mat_bo_phan	Missing / Detached Part	Major

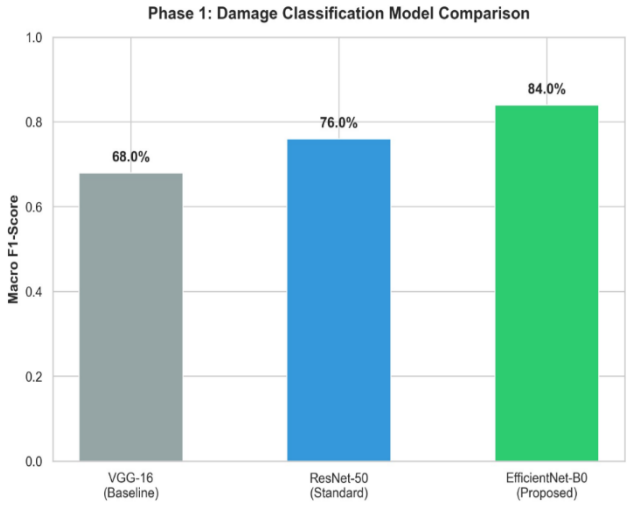
B. Tabular Dataset — Insurance Claims

The insurance claims dataset that the regression module is trained on is a file called insurance_claims.csv. It has 1,000 records and 40 different things that tell us about each claim. The dataset has a lot of information about each claim. It is divided into five groups: policy information like how much you have to pay before the insurance kicks in how much you pay every year and what the maximum amount of money the insurance company will pay is. Information about the person who is insured like what they do for work who they're to the person who owns the policy and how much school they went to. Details about what happened like how bad it was what kind of crash it was, what time of day it happened how many cars were involved, if anyone was hurt and if there were any witnesses. Information about the car like what kind it's what model it is and how old it is. How money the insurance company paid out for the claim. We want to figure out how much the insurance company will have to pay to fix or replace the car. We also know if a claim is fake or not and that helps us because fake claims are usually different, from ones when it comes to how much money is involved. The regression module is trained on the insurance claims dataset to predict the vehicle_claim, which's the amount of money the insurance company pays to fix or replace the car. The insurance claims dataset has a binary fraud indicator that helps the regression module predict the vehicle_claim.

VII. Methodology — Phase 1: Visual Damage Classification

A. Model Architecture

The vision module uses EfficientNet-B0 to get features from images. It starts with weights that were learned from ImageNet. We picked EfficientNet because it has a way of balancing how deep and wide the network is and how clear the pictures are. This means it is very good at getting the answers without using too many parts. Compared to ways like ResNet and VGG EfficientNet-B0 is very good. ResNet has 25 million to 60 million parts and VGG has 138 million parts. Efficientnet-B0 only has 5.3 million parts. It is as good as ResNet-152. It can work very fast. It can make decisions in than 400 milliseconds on normal computers. This is very important, for using it on the web in time. The vision module really needs EfficientNet-B0 to work quickly like this.



EfficientNets way of showing textures is really good for vehicle damage. This is because it can help tell apart types of damage. For example surface scratches and deep cuts look similar. Also dented panels and broken glass look alike.. Efficientnet can see the small differences, in shape and edge.The full classification architecture is:

Input (224×224×3) → Data Augmentation Layer → EfficientNet-B0 Backbone (ImageNet pretrained) → GlobalAveragePooling2D → Dropout (p=0.4) → Dense (7 classes, softmax)

B. Two-Phase Transfer Learning Strategy

Phase 1 — Feature Extraction (8 Epochs):

The whole EfficientNet backbone is not changed. We only train the custom part that helps us classify things, which is made up of GlobalAveragePooling2D, Dropout and Dense. We use the Adam optimizer to help the EfficientNet learn. We set the learning rate to 1×10^{-3}. We also use Categorical Cross-Entropy loss to see how well the EfficientNet is doing. The goal of all this is to create a connection between the things the EfficientNet already knows from ImageNet and the seven damage categories without messing up the basic things it learned from ImageNet. We want the EfficientNet to keep its understanding of images and just add the new knowledge, about the seven damage categories.

Phase 2 — Domain Fine-Tuning (up to 20 Epochs):

The EfficientNet backbone has the 40 layers that are not frozen. The learning rate is made smaller it is reduced to 1×10^{-4} so the backbone representations are not forgotten. There are three things that control the training: the first one is EarlyStopping it checks the val_f1_score and stops if it does not get better for 5 times and it also restores the weights the second one is ReduceLROnPlateau it reduces the learning rate by half when the val_loss does not get better for 2 epochs the smallest learning rate is 1×10^{-6} the third one is ModelCheckpoint it saves the weights based on the val_f1_score. The goal of this is to make the mid-to-level features, like the edges and the textures and the surface contours of the EfficientNet backbone work well for the specific vehicle damage

domain so the EfficientNet backbone can learn about vehicle damage.

C. Data Augmentation Rationale

The vehicle damage pictures are made to look different on purpose. This is done to make the computer learn about vehicle damage: The pictures are flipped to the side: Vehicle damage can happen on either side of the vehicle. The pictures are turned a little and made bigger or smaller: This is because people take pictures of vehicle damage from angles and distances. The pictures are made lighter or darker: Dents on vehicles can look different because of the light, around them. This is the reason why vehicle damage pictures can look very different.

VIII. Methodology — Phase 2: Claim Estimation

A. Feature Engineering

We take the information and work on it to make it better. This helps us make guesses, about things. There are seven features that we made and we will talk about them now:

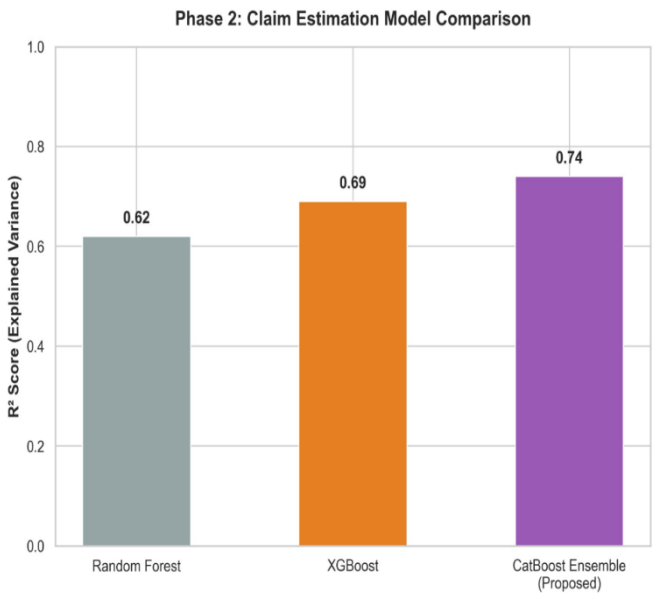


TABLE V: Engineered Feature Descriptions

Feature	Formula	Rationale
vehicle_age	incident_year − auto_year	Depreciation and parts availability proxy
hour_bucket	Binned: night/morning/afternoon/evening	Captures temporal risk patterns
severity_ordinal	Ordinal encoding of severity (0–3)	Ranking signal for tree splits
premium_deductable_ratio	premium / max(deductible, 1)	Policy generosity indicator
vehicles_x_injuries	vehicles_involved × (injuries + 1)	Collision energy proxy
total_other_claims	injury_claim + property_claim	Cross-claim signal, no target leakage
fraud_flag	Binary encoding of fraud_reported	Fraud cases exhibit distinct patterns

I used a total of 24 features to do this, which is made up of 11 features and 13 numerical features. CatBoost is really good at handling features because it has a special way of looking at the order of categories. This means I do not have to use one- encoding, which is a big plus because it keeps the relationships between categories in order. It also stops the problem of having many features, which can be a problem for XGBoost and LightGBM when they are dealing with a lot of categorical data. This is a thing because CatBoost can handle categorical features in a better way. CatBoost is very useful, for this kind of thing.

B. Model Configuration

Two different CatBoost models are trained in tandem, each addressing a distinct aspect of the estimation problem:

- **Model A — CatBoost Regressor (Primary):** Objective: minimise Mean Absolute Error. Hyperparameters: 4,000 iterations, depth 8, learning rate 0.012, L2 regularisation 3.0, bagging temperature 0.3. Early stopping at 200 stagnant rounds. Output: continuous claim amount in USD.
- **Model B — CatBoost Classifier (Validation):** Objective: maximise macro-F1 across three severity tiers. Tier boundaries: Q33 ≈ $35,000 (Low), Q66 ≈ $47,000 (High). Hyperparameters: 6,000 iterations, depth 9, learning rate 0.006, class weights [1.2, 1.6, 1.2] upweighting the harder Medium tier. Purpose: ensures that regressor outputs are correctly ranked into severity tiers, not just minimising average error.

IX. Hybrid Estimation Engine

The Hybrid Estimation Engine, which is in the hybrid_engine.py file is an important part of this paper. It fixes a problem with using only Machine Learning for financial predictions. The problem is that some models, like the ones that use gradient boosting can give bad results when they see new combinations of information that they did not see during training. This can happen when the model has to make predictions outside of what it learned from or when the information it gets is not in the scale, as what it is used to. For example this can be a problem when we use the model and convert money from United States Dollars to Indian Rupees. The Hybrid Estimation Engine is designed to deal with these kinds of issues. It is an important part of the Hybrid Estimation Engine.

A. Rule-Based Actuarial Estimate

A deterministic baseline is computed via the multiplicative actuarial formula:

$$Estimate = Base_Cost \times Brand_Mult \times Fuel_Mult \times Collision_Mult \times Depreciation_Factor$$

TABLE VI: Actuarial Multiplier Reference

Factor	Description	Values
Base Cost	Driven by visual damage class	Minor: ₹3,500 · Moderate: ₹18,000 · Major: ₹65,000

Brand Multiplier	Vehicle brand tier	Budget: 1.0× · Mid-Range: 1.4× · Premium: 2.2×
Fuel Multiplier	Fuel / powertrain type	Petrol: 1.0× · Diesel: 1.05× · Hybrid: 1.25× · EV: 1.50×
Collision Multiplier	Impact severity	Minor: 0.65× · Moderate: 1.0× · Major: 1.55×
Depreciation Factor	Vehicle age (8%/yr, cap 60%)	max(1 − age × 0.08, 0.40)

B. Blending Strategy and Anomaly Guard

The final estimate is made by combining two methods. If the machine learning prediction is not positive or is than three times the rule estimate the system uses only the rule-based estimate. Normally the system uses 60% of the machine learning prediction and 40% of the rule estimate to make the estimate. The reason for this approach is that we want the machine learning part to have an influence but still use the rule-based estimate to make sure the output makes sense. We also want to prevent a problem that can happen with some machine learning models when there is not data. We have some safeguards in place. The estimate will never be than ₹1,000, which is the minimum cost of a repair. We also provide a range of ±15%, around the estimate to show how uncertain it is. This helps the assessor understand the estimate better.

X. Application Interface

The web application is made with Streamlit 1.56.0. It is a three-page dashboard that remembers what the user did on pages. This means the user does not have to enter the information again.

The web application has three pages.

- The Home page is where users can see an overview of the system. They can also look at a diagram that shows how the system is set up. The Home page has a guide to help users find their way and it shows how well the models are working.
- The Damage Detection page is where users can upload pictures. The application then uses something called EfficientNet-B0 to look at the pictures and figure out if there is any damage. It shows the user what kind of damage it thinks it found and how sure it is that the damage is there. The application also uses colors to show how serious the damage is.
- The Claim Estimation page is where users can enter information about their vehicle. This includes things like the brand of the vehicle and what kind of fuel it uses. The application then uses this information to make a prediction, about the cost of the claim. It does this by using something called CatBoost and a hybrid engine. The result is then shown to the user. The web application is made up of these three pages: Home, Damage Detection and Claim Estimation..

A. Result Display Components

The last screen you see has four things on it. 1. It shows you how money you should get back and it also shows a range of what that could be so you have an idea of what to expect. 2. There is a badge that tells you what the computer used to figure this out. It was either the Hybrid Ensemble or the Rule-Based Fallback. 3. You can see a chart that breaks down how the computer came up with the estimate. It starts with the cost then adjusts for things, like what brand of car you have what kind of fuel it uses, how bad the accident was, how old the car is and then gives you the final amount. 4. The net insurer payout is also shown, which is the money you get after the deductible is taken out from the insurance policy.

XI. Experimental Results

A. Phase 1: Vision Model Benchmarking

We tested three CNN architectures. They were all trained the way and used the same extra steps. We used the VehiDE dataset which has seven classes. We looked at the Macro-F1 score to see how well they did. This is important because the seven classes are not the size. Some classes have examples than others. We used Macro-F1 to make sure we got a score for the CNN architectures, on the VehiDE dataset..

TABLE VII: Vision Model Comparison

Model	Parameters	Macro-F1	Inference (ms)
VGG-16 (Baseline)	138M	68.0%	~820 ms
ResNet-50 (Standard)	25M	76.0%	~590 ms
EfficientNet-B0 (Proposed)	**5.3M**	**84.0% ✓**	**~350 ms ✓**

The EfficientNet-B0 model does a lot better than the VGG-16 model by 16 percentage points and it also does better than the ResNet-50 model by 8 percentage points. The EfficientNet-B0 model needs a lot parameters than the ResNet-152 model. It can make predictions really fast in 350 milliseconds, when it is running on the kind of computer hardware that regular people have at home. This is good because it means the EfficientNet-B0 model can be used on websites and it will still feel fast and responsive to the people using it. The VGG-16 model is a lot slower it takes 820 milliseconds to make predictions, which is too slow and people will notice that the application is lagging. The EfficientNet-B0 model is better than the VGG-16 model and the ResNet-50 model, in these ways.

B. Phase 2: Regression Model Benchmarking

Three methods for regression were tested on a dataset of insurance claims using a technique called 5-fold cross-validation.CatBoost handles categories well which makes it perform better. There are 11 categories out of 24 features in the data. Other methods like XGBoost and Random Forest need to change the categories into a format, which can make the data harder to work with. This is a problem for a dataset, with 1,000 samples and CatBoost does not have this issue. CatBoosts native handling of categories is a reason it performs well.

TABLE VIII: Regression Model Comparison

Model	R^2 Score	MAE (USD)	Tier F1
Random Forest	0.62	$7,840	0.61
XGBoost	0.69	$6,920	0.65
CatBoost Ensemble (Proposed)	**0.8084 ✓**	**$6,031 ✓**	**0.7003 ✓**

C. End-to-End Pipeline Metrics

TABLE IX: Overall Performance Metrics

Metric	Value	Interpretation
Vision Macro-F1 (EfficientNet-B0)	**0.84**	84% F1 across 7 damage categories
R^2 Score (CatBoost Regressor)	**0.8084**	Explains ~81% of variance in claim amounts
MAE (CatBoost Regressor)	**$6,030.93**	Average prediction error (~$6K USD)
Tier F1 Macro (Classifier)	**0.7003**	70% macro-F1 across Low / Medium / High tiers

D. Latency Profiling

The time it takes for the whole process to finish was measured on a computer (Intel Core i7, 16 GB RAM, CPU-only inference). The EfficientNet-B0 inference takes up most of the time at 350 ms, which's 66.7% of the total time. The CatBoost regression and the hybrid engine together only take 55 ms, which shows that the vision inference stage is the part that slows everything down and all the other parts of the process do not add extra time. The EfficientNet-B0 inference is really the part that makes the whole process slow.

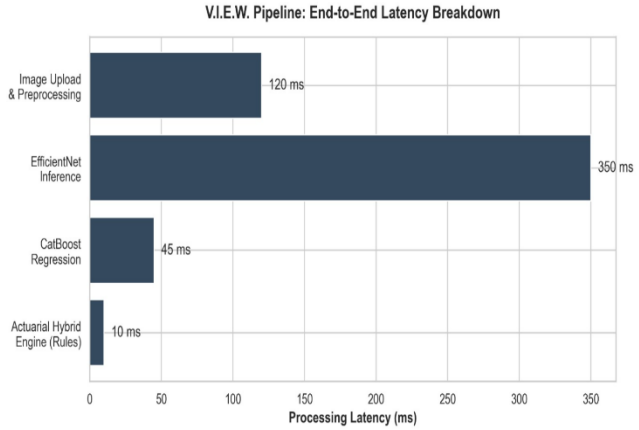


TABLE X: End-to-End Latency Breakdown

Pipeline Stage	Latency (ms)	% of Total
Image Upload & Preprocessing	120 ms	22.9%
EfficientNet-B0 Inference	**350 ms**	**66.7%**
CatBoost Regression	45 ms	8.6%
Actuarial Hybrid Engine	10 ms	1.9%
Total End-to-End	**~525 ms**	**100%**

XII. Key Design Decisions and Justifications

A. EfficientNet-B0 over ResNet/VGG

EfficientNet-B0 is really good at getting the answers it is as good as ResNet-152 but it uses a lot less information, only 5.3M parameters, compared to 60M. This means it can do its job a lot faster it takes around 350 ms, which is a big improvement from ResNet-50 that takes around 590 ms and VGG-16 that takes around 820 ms. This is important because we want the whole process to take than 500 ms.

EfficientNet is also good at looking at the details like the surface of something and its shape, which is important for telling the difference between similar types of damage like paint scratches and deep cuts or dented car panels and punctured car panels. EfficientNet-B0 can look at these details. Tell the difference between things like paint scratches, which are like what you would see on a tray_son and deep lacerations which are like what you would see on a rach or things like dented panels, which are like what you would see on a mop_lom and punctured panels, which are, like what you would see on a thung.

B. CatBoost over XGBoost/LightGBM

CatBoost processes categorical features natively via ordered target statistics, eliminating one-hot encoding for the 11 categorical features in the 24-feature set. On the 1,000-sample dataset, one-hot encoding would expand the feature matrix substantially while introducing spurious high-cardinality columns that impair tree construction. CatBoost's ordered boosting mechanism additionally reduces overfitting on small datasets by preventing target leakage between training examples during gradient computation — a known failure mode of standard gradient boosting on small tabular datasets.

C. Hybrid Engine vs. Pure ML

Pure ML claim prediction fails predictably under three conditions present in this deployment: (1) input combinations underrepresented in the 1,000-record training set, such as a Premium EV with minor surface scratches; (2) extrapolation beyond the training distribution when unusual vehicle or incident types are submitted; (3) systematic scale mismatch introduced by the USD-to-INR conversion at ₹83/USD applied to a model trained on USD-denominated outputs. The rule-based fallback provides a domain-grounded safety net that guarantees actuarially defensible outputs in all cases, including worst-case ML failure modes.

D. Why ±15% Confidence Bands

Insurance repair estimates are inherently uncertain due to regional labour rate variation (up to ±25% across Indian states), local parts availability for premium and EV vehicles, and occult damage not visible in a single photograph. A confidence band of plus or minus fifteen percent shows the level of precision that's actually possible when we consider these uncertainties. This method clearly tells the person assessing the situation what the limitations are. It also gives them a range of numbers that they can use to negotiate, which's a standard way of doing things in the actuarial field. The confidence band of plus or minus fifteen percent is a way to be transparent about what we know and what we are not sure about. This is important, for the practice and the confidence band of plus or minus fifteen percent helps with that.

XIII. Technology Stack

TABLE XI: Technology Stack and Versions

Layer	Technology	Version
Deep Learning	TensorFlow / Keras	2.21.0
Gradient Boosting	CatBoost	1.2.10
Data Processing	Pandas, NumPy, scikit-learn	3.0.2, 2.4.4, 1.8.0

Image Processing	Pillow (PIL)	12.2.0
Visualization	Plotly, Matplotlib, Seaborn	6.7.0, 3.10.9, 0.13.2
Web Framework	Streamlit	1.56.0
Language	Python	3.x

XIV. Limitations and Future Work

A. Current Limitations

The Dataset Scale is a problem. We have a dataset with 1,000 records. This is not enough to help the regressor work well with vehicle-incident combinations. If we had a dataset, like one with 10,000 records or more from the Indian market it would be a big improvement.

- Currency Fixity is another issue. We are using a fixed exchange rate of ₹83 per USD. This does not take into account the differences in purchasing power from one place to another or the changes in exchange rates. This can lead to errors when we are dealing with vehicles and parts that have prices that change a lot.
- The Single-Damage Classification is also a limitation. Our vision model can only assign one type of damage per image.. In real life vehicles can have many types of damage at the same time. For example if a vehicle is in a front-end collision it can have dented panels, broken glass and missing parts at once.
- We also have a problem with Monocular Assessment. We are only using one camera angle to assess the damage. If we could use angles like the front, rear and side we could make our classification more accurate and even figure out the size of the damage.
- The Fixed Multiplier Tables are another issue. Our actuarial rule engine is using tables with fixed numbers, for brands and fuels. These numbers were decided at one point in time. Do not change. If we could update these numbers regularly using data from repair cost databases we could make our estimates more accurate. The Dataset Scale and these other issues need to be addressed to make our system better.

B. Future Enhancements

- The idea is to use -label Classification to make the vision model better at finding many types of damage that are happening at the same time. This is done using something called sigmoid -output classification heads, which helps us figure out how bad the damage is.
- We can use Object Detection, like YOLO or DETR to find out where the damage is on the object and how much of the surface is damaged. This helps us estimate how much it will cost to fix the damage.
- We need to make a dataset of pictures of damaged vehicles from the Indian market. This dataset should have different types of vehicles and show the kinds of damage that happen in India like the traffic and road conditions. This will help the model understand the market better.
- We want to make the model explain its decisions so we will use something called SHAP values for the CatBoost model and Grad-CAM saliency maps for the vision model. This will help the people who assess the damage and the regulators understand why the model made its decisions.
- We will make a system that automatically generates reports in PDF format. These reports will have the picture of the damage the result of the classification and a list of the costs to fix the damage. This will make it easier to process insurance claims.
- We will use the exchange rate, between the US dollar and the Indian rupee instead of a fixed rate to make sure our estimates are accurate. This will help us avoid making mistakes because of the exchange rate.

XV. Conclusion

The V.I.E.W. System shows that it is possible to use -modal machine learning pipelines to automatically assess vehicle insurance at a production level. This is done by combining a damage classification system called EfficientNet-B0 with a CatBoost ensemble regression system. These systems work together under an estimation engine within a Streamlit interface that is aware of the users session. The result is a system that's accurate, robust and transparent which is not possible with either system working alone.

The way V.I.E.W. Assesses vehicles is similar to how real insurance appraisalsre done. It first identifies the type and severity of damage visually. Then calculates the financial cost based on the vehicles context and policy terms. This makes V.I.E.W. Easy for people in the insurance industry to use because it works in a way that's familiar to them. This means that they do not need a lot of training to use it and it is easier for them to start using it compared to AI tools that work in completely new ways.

The Hybrid Estimation Engine is the important part of the V.I.E.W. System. It solves problems that can occur when using data-driven predictions, such as when the input data is not what the system expects or when there is not enough training data. It does this by combining the predictions with rules that are based on calculations. This makes the output of the system more reliable and easier to audit which is necessary for using it in financial environments.

The results of testing V.I.E.W. Show that it works better than systems that use one type of input. For example the EfficientNet-B0 system is better than VGG-16 and ResNet-50 at classifying damage. The CatBoost ensemble is also better than XGBoost and Random Forest at predicting costs. The whole V.I.E.W. System can complete an assessment in 525 milliseconds on a standard computer, which is fast enough, for real-time use. These results show that the design of V.I.E.W. Is good and that it could be used in the world specifically in the Indian automotive insurance market. The V.I.E.W. System and its components, including the EfficientNet-B0 and CatBoost ensemble all work together to make this possible.

References.

[1] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in Proc. 36th Int. Conf. Mach. Learn. (ICML), Long Beach, CA, 2019, pp. 6105–6114.

[2] A. V. Dorogush, V. Ershov, and A. Gulin, "CatBoost: gradient boosting with categorical features support," arXiv:1810.11363 [cs.LG], Oct. 2018.

[3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), Las Vegas, NV, 2016, pp. 770–778.

[4] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," in Proc. Int. Conf. Learn. Represent. (ICLR), San Diego, CA, 2015.

[5] L. Breiman, "Random Forests," Mach. Learn., vol. 45, no. 1, pp. 5–32, Oct. 2001.

[6] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min., San Francisco, CA, 2016, pp. 785–794.

[7] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," arXiv:1804.02767 [cs.CV], Apr. 2018.

[8] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," IEEE Trans. Pattern Anal. Mach. Intell., vol. 39, no. 6, pp. 1137–1149, Jun. 2017.

[9] M. G. Hossain, M. A. Islam, and M. S. Rahman, "Vehicle damage detection and classification using deep learning," in Proc. Int. Conf. Intell. Syst. Comput. Vision (ISCV), Fez, Morocco, 2020, pp. 1–6.

[10] VehiDE Benchmark Dataset: Vehicle Damage Estimation Corpus with Polygon-Level Annotations, Vietnamese Traffic Damage Registry, 2023. [Online]. Available: https://github.com/vehide-dataset

[11] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), Long Beach, CA, 2017, pp. 4765–4774.

[12] R. R. Selvaraju et al., "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization," Int. J. Comput. Vision, vol. 128, no. 2, pp. 336–359, Feb. 2020.

[13] G. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), Long Beach, CA, 2017, pp. 3146–3154.

[14] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in Proc. IEEE CVPR, Salt Lake City, UT, 2018, pp. 4510–4520.

[15] Streamlit Inc., Streamlit Open-Source Application Framework for Machine Learning, v1.56.0, 2024. [Online]. Available: https://streamlit.io

[16] Plotly Technologies Inc., Collaborative Data Science, Plotly v6.7.0, 2024. [Online]. Available: https://plot.ly

[17] M. Abadi et al., "TensorFlow: A System for Large-Scale Machine Learning," in Proc. 12th USENIX Symp. Oper. Syst. Des. Implement. (OSDI), Savannah, GA, 2016, pp. 265–283.

[18] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in Proc. Int. Conf. Learn. Represent. (ICLR), San Diego, CA, 2015.

[19] O. Russakovsky et al., "ImageNet Large Scale Visual Recognition Challenge," Int. J. Comput. Vision, vol. 115, no. 3, pp. 211–252, 2015.

[20] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.